

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Shahanaj c	Reg. No.:	192372292
Section / Batch:	CSE AI	Date:	24/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Code of Conduct

I _____shahanaj / 192372292_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____shahanaj_____

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive

Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation ; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				

Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Program:

```
import heapq
```

```
class Graph:
```

```
def __init__(self, vertices):
```

```
self.V = vertices
```

```
self.edges = []
```

```
self.adj = [[] for _ in range(vertices)]
```

```
def add_edge(self, u, v, w):
```

```
self.edges.append((w, u, v))
```

```
self.adj[u].append((v, w))
```

```
self.adj[v].append((u, w))
```

```
def prims_mst(self):
```

```
visited = [False] * self.V
```

```
min_heap = [(0, 0)]
```

```
total_cost = 0
```

```
mst_edges = []
```

```
parent = [-1] * self.V
```

```
while min_heap:
```

```
weight, u = heapq.heappop(min_heap)
```

```
if visited[u]:
```

```

        continue
    visited[u] = True
    total_cost += weight
    if parent[u] != -1:
        mst_edges.append((parent[u], u, weight))

    for v, w in self.adj[u]:
        if not visited[v]:
            heapq.heappush(min_heap, (w, v))
        parent[v] = u

    return mst_edges, total_cost

def find(self, parent, i):
    if parent[i] != i:
        parent[i] = self.find(parent, parent[i])
    return parent[i]

def union(self, parent, rank, x, y):
    if rank[x] < rank[y]:
        parent[x] = y
    elif rank[x] > rank[y]:
        parent[y] = x
    else:
        parent[y] = x
        rank[x] += 1

def kruskals_mst(self):
    result = []
    i, e = 0, 0
    self.edges = sorted(self.edges, key=lambda item: item[0])

    parent = list(range(self.V))
    rank = [0] * self.V

    while e < self.V - 1 and i < len(self.edges):
        w, u, v = self.edges[i]
        i += 1
        x = self.find(parent, u)
        y = self.find(parent, v)

        if x != y:
            e += 1

```

```

result.append((u, v, w))
self.union(parent, rank, x, y)

total_cost = sum(w for u, v, w in result)
return result, total_cost

```

```

def print_mst(name, mst_edges, total_cost):
    print(f"\n--- {name} MST ---")
    for u, v, w in mst_edges:
        print(f'Edge: {u} - {v} Weight: {w}')
    print(f'Total Cost: {total_cost}')

```

```

def main():

```

```

    V = int(input("Enter number of vertices: "))
    E = int(input("Enter number of edges: "))
    g = Graph(V)

```

```

    print("Enter each edge as: u v weight")
    for _ in range(E):
        u, v, w = map(int, input().split())
        g.add_edge(u, v, w)

```

```

    prim_edges, prim_cost = g.prim_mst()
    print_mst("Prim's", prim_edges, prim_cost)

```

```

    kruskal_edges, kruskal_cost = g.kruskals_mst()
    print_mst("Kruskal's", kruskal_edges, kruskal_cost)
    print("\n--- Comparative Analysis ---")
    print(f'Prim's Total Cost: {prim_cost}')
    print(f'Kruskal's Total Cost: {kruskal_cost}')

```

```

    if prim_cost == kruskal_cost:
        print("Both algorithms produced MSTs of equal total weight (as expected).")
    else:
        print("Discrepancy detected — check graph connectivity or input.")

```

```

    print("\n--- Insights ---")
    print("1. Prim's Algorithm grows the MST one vertex at a time using a priority queue;")
    print("it is generally more efficient for DENSE graphs (Time:  $O(E \log V)$  with heap).")
    print("2. Kruskal's Algorithm sorts all edges and uses Union-Find to avoid cycles;")

```

```

print(" it performs better on SPARSE graphs (Time: O(E log E)).")
print("3. Edge Selection Strategy:")
print(" - Prim's: Selects the minimum weight edge connected to the growing tree.")
print(" - Kruskal's: Selects the global minimum weight edge that doesn't form a cycle.")
print("4. Suitability:")
print(" - Use Prim's for graphs represented as adjacency matrices/lists with many edges.")
print(" - Use Kruskal's for edge-list representations or when the graph is sparse.")

```

```

if __name__ == "__main__":
    main()

```

OUTPUT:

The screenshot shows the SIMATS Online Programming Lab interface. The header includes the SIMATS ENGINEERING logo, the lab name, and the user's name (CHERUVU SHAHANAJ) with a dropdown arrow. The main section is titled "ASSESSMENT CO4-AT3-COMPARATIVE ANALYSIS". Below this, there are tabs for "Language: Python C C++ Java". The "QUESTIONS" section on the left shows a question titled "Q1 Prim's vs Kruskal's Algorithm (MST Construction)" with a green checkmark and "100 marks". The main content area displays the question details: "Q1 Prim's vs Kruskal's Algorithm (MST Construction) 100 marks". The question text is: "Given a graph, construct MST using Prim's and Kruskal's methods. Compare edge selection strategies. Analyze efficiency in dense vs sparse graphs. Generate insights on MST algorithm suitability." Below the question, there are three sections: "Your Code" with a code editor showing a Python class for graph operations, "Input" with a text area containing a graph representation, and "Output" with a text area showing the prompt for the MST.

SIMATS Online Programming Lab
DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292RA

ASSESSMENT CO4-AT3-COMPARATIVE ANALYSIS

Language: Python C C++ Java

QUESTIONS

Q1
Prim's vs Kruskal's Algorithm (MST Construction)
100 marks

1/1 answered

Q1 Prim's vs Kruskal's Algorithm (MST Construction) 100 marks

Given a graph, construct MST using Prim's and Kruskal's methods. Compare edge selection strategies. Analyze efficiency in dense vs sparse graphs. Generate insights on MST algorithm suitability.

Your Code

```

1 import heapq
2
3 class Graph:
4     def __init__(self, vertices):
5         self.V = vertices
6         self.edges = []
7         self.adj = [[] for _ in range(vertices)]
8
9     def add_edge(self, u, v, w):
10        self.edges.append((w, u, v))
11        self.adj[u].append((v, w))
12        self.adj[v].append((u, w))

```

Input

```

5
6
0 1 2
0 3 6

```

Output

```

Enter number of vertices: Enter
number of edges: Enter each edge
as: u v weight
--- Prim's MST ---

```